Qwen

# From Schemata to Systems: A Unified Knowledge Graph and CI-Driven Framework for Multi-Engine Game Development

## Foundational Architecture: The Schema-First Paradigm

The foundational architectural paradigm for the "Conker: Live & Uncut" fan project is a deliberate departure from traditional, document-centric software development methodologies. Instead of relying on informal communication and implicit understanding, the framework establishes a **schema-first** approach where every significant artifact—be it code, configuration, documentation, or assets—is governed by a formal, machine-verifiable contract [27]. This paradigm is designed to create a unified, self-documenting codebase that prioritizes long-term maintainability and enables highly effective AI-assisted development [1] [16]. The entire repository structure, from engine-specific implementations to supporting toolchains, is being built upon a set of shared, validated data structures that serve as the single source of truth for the project's intent and constraints. This approach ensures that all components, regardless of their language or engine, speak a common language defined by these schemas, which act as both a contract and a blueprint for implementation.

A central tenet of this philosophy is the strict anchoring of abstract architectural objectives to concrete, near-term, and shippable implementations. For instance, the high-level goal of achieving long-term maintainability is realized not through a vague promise but through the immediate creation of a `repo_index_generator.rs` tool in the `Tools/indexing` directory [25]. This CLI, when implemented, will crawl the repository and generate a structured JSON index of files, languages, and declared symbols, directly translating the objective of improved discoverability into a tangible artifact [26]. Similarly, the goal of enforcing cross-engine consistency is addressed by defining and validating a `mission.graph.schema.json` for multiplayer objectives, which provides a clear contract for the corresponding Unreal Engine C++ and Godot GDScript implementations [24] [50]. This practice of grounding strategy in executable steps prevents the framework from becoming purely theoretical and ensures that progress is continuously manifested in

working parts of the project. The explicit exclusion of speculative or non-contract-based enhancements reinforces this commitment to rigor, focusing efforts solely on machine-verifiable constraints that demonstrably improve either code quality or documentation quality [27].

The schema-first paradigm extends beyond just gameplay logic to encompass the entire development lifecycle, including documentation, asset pipelines, and even legal compliance. The repository's top-level documentation in the `Docs` directory is treated as a structured set of inputs for both human collaborators and AI systems [1]. Documents like `01_Game_Overview.md` and `TechDesign/01_Engine_Choice_UE5_Unity_Godot.md` are intended to be progressively formalized into schemas that define game pillars, tone tags, and system-to-engine mappings [29] [35]. This allows a developer or an AI to query the project's design intent programmatically rather than sifting through prose. The asset pipeline, housed in the `Assets_Source` directory, is treated as the canonical, legal origin for all content, with manifest files (`ModelsManifest.json`, `TexturesManifest.json`, etc.) serving as schemas that link source assets to their platform-specific costs and constraints . This creates a provable link from a raw `.fbx` model to its final byte cost in a PS1 ROM, enabling automated budget enforcement [35]. Even the project's governance is captured schematically; the `LICENSE` file's terms are intended to be mapped to specific directories and KG nodes, allowing a small checker tool to ensure that no original IP is accidentally distributed while permitting reuse of the fan-made tooling .

This architectural choice is fundamentally about reducing ambiguity and increasing verifiability at every level. By treating the codebase as a collection of interconnected, schema-driven objects, the framework transforms development from a series of ad-hoc decisions into a process of constructing and validating a complex, consistent system. The reliance on formal proofs and mathematical models, primarily for validation purposes, further solidifies this foundation [38]. A build fails if it violates a formally defined invariant, such as a budget constraint or a determinism requirement, providing an unambiguous signal that a change has broken a core principle of the project [19]. This contrasts sharply with approaches that rely on subjective reviews or informal checks. The ultimate goal is to create a system where the correctness of a change can be determined by examining the schema, the mathematical model, and the CI logs, rather than by asking a human expert. This systematic approach, inspired by practices from AAA game programming and modern DevOps, aims to scale the project's complexity without a corresponding increase in cognitive load or risk of regression [2] [22]. It represents a mature engineering strategy for managing a large-scale, multi-faceted project,

particularly one driven by both human and AI contributors, by building a robust, self-enforcing structure from the ground up.

## Knowledge Graph Integration: The Central Nervous System for Code and Context

The Knowledge Graph (KG) serves as the conceptual and practical backbone of the "Conker: Live & Uncut" research framework, acting as the central nervous system that connects disparate components of the project into a navigable, semantic whole. Its primary function is to transform the repository from a static hierarchy of files and folders into a dynamic graph of interrelated concepts, making it possible for both humans and AI systems to understand and traverse the codebase based on meaning rather than just file paths [45] [46]. Every significant element—from a C++ header file like `CLUCharacterBase.h` to a Lua script like `heist_round_logic.lua`, a design document like `02_Multiplayer_Heist.md`, and a Rust tool like `repo_index_generator.rs`—is registered as a distinct entity within the KG, typically as a `SystemNode` or a `FeatureEntry` [42] [47]. These entities are not merely named; they are richly annotated with metadata that captures their role, dependencies, location, and relationship to other parts of the system. This structured indexing is the key to unlocking the full potential of the schema-first approach, providing the context necessary for high-fidelity AI code generation and efficient human navigation [43].

The mechanism for implementing the KG involves populating and maintaining a set of JSON files that act as the master index. Key among these are `knowledgegraphsystems.json` and `knowledgegraphfeatures.conker.json`. Each entry in these files, or `SystemNode`, would represent a major subsystem or component. For example, the UE5 character base would be indexed as a node with a unique ID like `systems.ue5.conker.core.character`. This node would contain crucial metadata: a `tags` array with values like `UE5`, `Deterministic`, and `Netcode`; a `path` field pointing to `Engine/Unreal/Source/Public/Core/CLUCharacterBase.h`; and links to related schemas, such as the `CharacterMechanic` schema that defines its state and input model . Similarly, the Heist multiplayer mode would be a node (`systems.ue5.conker.multiplayer.heist`) linked to its public and private headers, the `heist.gamemode.schema.json` that dictates its rules, and potentially to test scenarios that exercise its logic . This tagging and linking strategy is critical, as it

allows queries to be executed against the graph. An AI could be prompted to "find all UE5 systems tagged `Netcode`," or a developer could ask for a list of all features related to the `heist` mode, receiving a curated list of relevant nodes and their associated files and schemas.

This KG-driven navigation provides immense value in managing a complex, multi-engine project. It bridges the gap between different technology stacks by exposing them through a common semantic layer. For instance, a Godot script implementing a character mechanic, `conker_character.gd`, would be indexed as another `SystemNode` with tags like `GodotFrontend` and `Deterministic`, and it would be explicitly linked to the same `CharacterMechanic` schema that the UE5 `CLUCharacterBase` and the backend Rust ECS implementation conform to . This makes it immediately clear that these disparate implementations are part of a cohesive, multi-platform system. The KG also serves as a powerful tool for discovery and onboarding. Instead of guessing where to find the code for a specific feature, a new contributor or an AI agent can query the KG using natural-language prompts translated into graph queries. The proposed `gamemodeai-kg` CLI tool would facilitate this, allowing users to perform path and scope queries that understand relationships across both Rust and UE5 systems . For example, a query for the "path from `systems.retro.nintendoor64.starzip.layout` to `systems.ue5.conker.multiplayer.heist`" could reveal the chain of adapters and build recipes required to transport a concept from one engine to another, demystifying the project's architecture .

The implementation of the KG is an iterative process anchored in concrete actions. The initial step, identified as a high-impact objective, is to create the foundational KG index files (`knowledgegraphsystems.json` and `knowledgegraphfeatures.conker.json`) by manually enumerating the key systems described in the initial repository structure . This seeds the graph with the most critical nodes. Subsequently, the `repo_index_generator.rs` tool becomes essential for scaling this effort. This Rust CLI would be designed to crawl the repository tree, using parsers like `syn` for Rust, `tree-sitter` for multiple languages, or regular expressions for simpler symbol extraction . It would analyze each file, identify classes, functions, and other symbols, and enrich the corresponding KG nodes with this detailed information. The output would be a comprehensive `repo_index.json` file, which could then be used by the `gamemodeai-kg` tool to enable fine-grained navigation down to the level of a specific function, allowing a prompt like "jump to the implementation of `HandleVaultOpened` in `CLUHeistGameMode.cpp`" to be resolved directly . This combination of manual seeding for strategic components and automated enrichment for granular details creates a scalable and robust KG infrastructure. Furthermore, the KG is not just for navigation; it is a living document that reflects the project's evolving state. As

new systems are added or existing ones are refactored, their KG entries are updated, ensuring the map always reflects the territory. This continuous synchronization between code and knowledge graph is a cornerstone of the framework's ability to maintain long-term architectural alignment.

## Orchestration and Automation: Translating Principles into Action

The principles of schema-first design and knowledge-graph navigation are given life and enforceability through a sophisticated orchestration and automation layer. This layer acts as the bridge between the abstract architectural vision and the concrete development workflow, translating policies and constraints into automated, repeatable actions that govern the entire project lifecycle. At its core, the system leverages GitHub Actions to automate building, linting, and testing across the diverse set of technologies involved, including Unreal Engine 5, Godot, Unity, Rust, and MATLAB [25] [26]. The `.github/workflows/build.yml` file is the central declaration of this CI/CD pipeline, defining the jobs and steps that run on every push or pull request [7]. However, the framework's approach goes far beyond standard CI/CD by embedding rigorous, mathematically-defined validation gates directly into these workflows. This ensures that the project's core architectural invariants—such as budget limits, determinism requirements, and ABI stability—are not just guidelines but hard constraints that cannot be violated without failing the build [19] [38].

The most critical function of this orchestration layer is to serve as a gatekeeper for the project's integrity. The research directive specifies that mathematical models should be developed primarily for validation and gating purposes, forming a robust defense against regressions and architectural drift [19]. For example, a CI job would not only compile the code but also execute a "Hardware-Aware Budget Planner" that uses predefined size formulas for assets and code modules to compute the total ROM and VRAM usage . If this computed total exceeds the predefined budget polytope—a feasible region defined by linear inequalities—the build fails immediately, preventing a costly mistake from reaching the main branch [35]. Similarly, for networked game modes, a CI check would validate that any changes to the game logic adhere to the determinism invariants formalized in a mathematical model, ensuring that scripted match replays remain consistent across builds [10]. This shift from passive monitoring to active, policy-driven enforcement is a defining characteristic of the framework. It moves quality assurance

from a post-facto review process to a proactive, automated one, deeply integrated into the daily development cycle.

The underlying orchestration model appears to be inspired by concepts like "Sonia," a system for terminal-less orchestration of computational tasks [29]. In this model, a central orchestrator manages sessions and coordinates the execution of a series of deterministic command-line interfaces (CLIs). Each CLI operates on well-defined inputs and produces a structured output, often in JSON format, adhering to a formal contract like an `ArtifactSpec` [14] [32]. This design promotes modularity and composability. For instance, the overall build process might be orchestrated by a `sonia-core`-like binary that invokes a sequence of specialized CLIs: `gamemodeai-build` to compile UE5 modules and Unity projects, `repo_index_generator` to update the knowledge graph, and a custom scenario builder CLI to run tests for the Heist mode [29]. This approach allows for complex, multi-step operations to be managed as a single, atomic unit. The `SessionProfile` concept is central to this model, representing the state of a particular task or build, containing information like the active crate, branch, and applied invariants . By treating each development action as a session with a clearly defined profile, the system can ensure that all subsequent tool invocations operate under the correct context and constraints.

To support this model, the framework relies on a suite of custom-built Rust CLIs that expose the project's internal logic and data in a machine-readable way. These tools are not mere scripts; they are first-class citizens in the ecosystem, registered as `SystemNodes` in the knowledge graph and governed by the same schemas as the rest of the codebase . Examples include `clr_unpack_rs` for processing Xbox-style archives, `n64_asset_converter` for converting retro assets, and `net_latency_analysis.m` (which would likely be wrapped by a CLI for integration into the CI flow) for analyzing networking parameters [37]. These CLIs are designed to have clear subcommands (`inspect`, `list`, `extract`), consume and produce structured JSON, and fail predictably when their contracts are violated. This JSON-RPC-like envelope makes them ideal for programmatic consumption by the orchestrator . The orchestration layer thus becomes a directed acyclic graph (DAG) of tool invocations, where the output of one CLI serves as the input for the next. The mathematical reasoning developed for build graphs and dependency analysis can be reused here to ensure there are no circular dependencies and to estimate the cost of a given session, preventing runaway resource usage [29]. This fully automated, policy-driven workflow is what ultimately translates the ambitious architectural goals into a reliable and scalable development process.

# Engine-Specific Implementations and Cross-Platform Consistency

The research framework strategically assigns distinct roles to Unreal Engine 5 (UE5), Godot, and Unity, leveraging each engine's unique strengths to create a balanced and efficient development pipeline. UE5 is designated as the primary production target, tasked with delivering the final, high-fidelity version of the game [2]. Its focus is on performance-critical systems, including complex rendering, physics, and the authoritative server-side replication for multiplayer modes. Godot is positioned as the preferred rapid-prototyping environment, valued for its lightweight nature and fast iteration cycle, making it ideal for quickly testing and refining gameplay mechanics and rule sets before committing to the heavier UE5 development pipeline [36]. Unity serves as a secondary, experimental target, providing a sandbox for parallel prototyping and exploring alternative implementation patterns without disrupting the primary workflow [3]. Despite these differing roles, all three engines are kept functionally consistent and interoperable through a thin, generated bridge layer backed by a shared set of Rust-based schemas and a central knowledge graph [2] [17]. This ensures that a mechanic or rule defined in Godot can be reliably transplanted to UE5, guided by the unambiguous contracts established in the schema layer.

For Unreal Engine 5, the alignment strategy is to treat its components as first-class entities within the Knowledge Graph. Every major C++ class and module, from `CLUCharacterBase` to the various `CLU*GameMode` classes, is registered as a `SystemNode` with explicit metadata, including tags like `UE5`, `Deterministic`, and `Netcode`, along with direct links to their implementation files and corresponding schemas . This deep integration allows the entire UE5 project skeleton to be navigated and understood semantically. The research objectives for the UE5 side involve formalizing its subsystems according to the mathematical models developed for the project. For example, the `CLUCharacterBase` class's movement and state updates would be modeled as a deterministic step function, mirroring the formalism used for the backend Rust ECS [33]. Similarly, the `CLUHeistGameMode` would be driven by a `heist.gamemode.schema.json` that defines its phases, objectives, and transitions as a state machine, moving away from hard-coded logic towards a data-driven design . This formalization allows CI tools to verify properties of the UE5 code, such as ensuring that the objective DAG has no unreachable states or that state transitions respect certain monotonicity constraints, thereby enforcing design correctness at a fundamental level.

Godot's role as a rapid-prototyping sandbox is supported by a strategy of treating its GDScripts as front-end implementations of abstract, engine-agnostic schemas. The `conker_character.gd` script, for instance, is not an isolated piece of code but is intended to be a faithful implementation of a shared "ConkerMechanicModel" defined in a JSON schema . This model would specify the character's state space, input set, and observable outputs, providing a contract that both the Godot script and the backend Rust ECS must satisfy. This approach enables powerful cross-engine equivalence testing. A test harness could drive identical input traces through the Godot prototype and the Rust simulation, then compare the resulting observable trajectories (e.g., position, animation state) over time . Any significant deviation between the two could indicate an error in the Godot implementation or a misunderstanding of the model, allowing issues to be caught early in the low-cost prototyping phase. This method ensures that the speed of iteration in Godot does not come at the cost of fidelity to the project's core mechanics. The `heist_mode.gd` and `war_mode.gd` scripts follow the same pattern, being driven by their respective mission and state-machine schemas, which are validated by external Rust tools to prevent logical errors before the logic is ever considered stable enough for porting to UE5 .

Unity, as the experimental target, follows a similar alignment strategy centered on the C Application Binary Interface (ABI) bridge. The goal is to create a stable C API that acts as the single point of contact between the performance-critical, engine-agnostic world logic (written in Rust/C++) and the Unity C# scripting environment . Thin, generated wrappers around this C ABI would be created to translate Unity events into schema-compliant input frames and to expose read-only world state for rendering and UI purposes . This isolates the core logic from engine-specific details and ensures that any experiments conducted in Unity, such as different networking patterns or user interface layouts, are built upon a stable and predictable foundation. The configuration for Unity, including world settings and mode definitions, would be driven by JSON schemas that mirror those used by the other engines, ensuring that a `WorldConfig` loaded by Unity's C# code is structurally identical to the one processed by the Rust backend . This schema-driven approach to experimentation prevents the creation of divergent, hard-to-maintain codepaths and keeps all experimental work within the bounds of the project's established contracts. The table below summarizes the distinct roles and alignment strategies for each engine target.

| Feature | Unreal Engine 5 (UE5) | Godot | Unity |
|---|---|---|---|
| Primary Role | **Primary Production Target:** High-fidelity release build focusing on performance and final polish [2] . | **Rapid Prototyping:** Fast iteration on gameplay mechanics and rule sets [36] . | **Experimental Target:** Parallel prototyping and exploration of alternative patterns [3] . |
| Core Technology | C++ | GDScript | C# |
| Alignment Strategy | Deep KG integration; formalization of UE5 systems into deterministic algebraic models . | Treat scripts as front-ends for abstract schemas; enable cross-engine equivalence testing . | Use a stable C ABI bridge to isolate core logic; drive config with shared schemas . |
| Key Artifacts | `CLU<em>` C++ classes, `</em>.uproject`, `.Target.cs` files [2] . | `*.gd` scripts, `project.godot`, `scenes/` directory [36] . | `Assets/Scripts/` C# classes, `ProjectSettings/` directory . |
| Validation Focus | Formal verification of state machines and algebraic models via CI checks . | Cross-trace comparison with Rust/ECS equivalent to ensure mechanical fidelity . | Ensuring C# wrappers correctly interface with the C ABI and honor schema contracts . |

This tiered approach, combined with a strong schema and KG foundation, allows the project to leverage the best tool for each job while maintaining a coherent, consistent, and ultimately achievable vision for the final product.

# Mathematical Modeling for Validation and Design Guidance

The research framework places a strong emphasis on the development of mathematical models, but with a clear and pragmatic strategy: these models are primarily conceived and implemented for rigorous validation and CI gating. Their utility is then extended secondarily to provide quantitative guidance for design decisions, thereby avoiding the creation of redundant, "soft" mathematical layers and ensuring all quantitative reasoning is rooted in provable, machine-checkable constraints [19] [38] . This "validation-first, guidance-second" approach ensures that the project's most critical assumptions are actively defended by automated checks, while simultaneously providing valuable, data-driven insights to developers and designers. The mathematical constructs range from simple inequalities for budget constraints to complex state-space analyses for determinism and networking latency.

The primary application of mathematics is in the creation of validation gates within the CI/CD pipeline. These gates enforce the project's core architectural invariants by translating them into executable checks.

- **Budget Constraints:** Hardware limitations such as ROM size, VRAM, and CPU cycles are formalized as a system of linear inequalities, defining a feasible region or "budget polytope" [35]. The Hardware-Aware Budget Planner uses cost functions derived from asset manifests (for models, textures, and audio) and code module sizes to calculate the total resource consumption . A CI job would fail if the calculated sum for a given commit exceeds the predefined limits, preventing developers from inadvertently introducing performance regressions [35].
- **Determinism Invariants:** For a networked game, the ability to reproduce gameplay sequences is paramount for debugging and testing. The framework models the game world's evolution as a deterministic step function, $W_{t+1}=S_k\circ\cdots\circ S_1(W_t,I_t)$, where the world state $W$ at the next tick is a function of the current state and player inputs $I$ [10] [33]. CI checks would enforce this property by running scripted scenarios (defined in `ScenarioSpec`) on different build versions and comparing the resulting world snapshots. Any divergence would trigger a failure, ensuring that gameplay remains deterministic across updates [29].
- **Networking Latency and Tick Rate:** Networking configurations are notoriously difficult to tune. The framework addresses this by using basic queueing theory and physics-like formulas to model latency [29]. Formulas for expected delay ($D\approx\mathrm{RTT}/2+\text{interpolation window}$) and jitter envelopes are used to define safe operating regions for different tick rates and player counts . A CI check could validate that the chosen tick rate and snapshot compression settings for a multiplayer mode fall within these safe regions, preventing network-related bugs from entering the codebase [29].
- **ABI Stability:** To ensure that the C ABI bridge between Rust and other engines remains stable, formal guards can be implemented. These guards might use checksums or hashes of exported function signatures and data structures, comparing them against a known-good baseline. A CI check would fail if a change to the Rust backend causes these hashes to change, preventing breaking changes to the contract exposed to UE5, Unity, or other clients [12].

Once these models are proven and operational as validation gates, their equations and principles are repurposed to provide direct design guidance, improving both code and

document quality . This reuse eliminates the need to develop separate, less rigorous models for design-time use.

- **Stealth Visibility Curves:** The same mathematical formulas used in a CI gate to validate that a character's detection state is a pure function of its position and posture can be embedded in a level editor or a documentation guide. A designer could use this formula to score potential stealth routes in a map, ensuring they meet the desired difficulty curve before they are even built .
- **Mission DAG Complexity Metrics:** The Mission Directed Acyclic Graph (DAG) math, used in CI to check for unreachable objectives or excessive branching, provides metrics like branching factor and path count . These same metrics can be visualized in a GDD or a design document to help a game designer balance the replayability and complexity of a multiplayer mode's objective flow .
- **Networking Budget Planner:** The networking latency formulas, once validated, can be turned into an interactive UI or a scoring metric. When a developer proposes a new feature for a multiplayer mode, the system could use the formulas to calculate its impact on latency and bandwidth, providing a quantitative "score" that helps in balancing feature richness against network performance .
- **Reputation Vectors for Narrative Progression:** The reputation vector model, $r_k = r_0 + \sum_e r_e$, used to track narrative consequences in a branching story, can be both a CI check (ensuring vector inequalities hold after a merge) and a design tool. A writer could use this model to plan the moral or factional consequences of different choices in a campaign, with the vector components representing different standing levels [13].

By adopting this dual-purpose strategy, the framework ensures that its quantitative reasoning is grounded in reality. Every number and formula used for design guidance has been previously vetted by an automated test, lending it credibility and trustworthiness. This approach elevates the discussion from subjective opinions ("this feels too slow") to objective, data-driven arguments, fostering a more rigorous and productive development environment.

## Synthesis and Strategic Implementation Roadmap

The research framework for "Conker: Live & Uncut" presents a holistic and systematically engineered approach to managing a complex, multi-engine game project. Its core strength lies in the tight integration of three pillars: a schema-first paradigm that treats the entire repository as a collection of formal, machine-verifiable contracts; a knowledge

graph that serves as a central, navigable index of all project components; and a robust orchestration layer that automates the enforcement of architectural invariants. This architecture is not merely a theoretical construct but is explicitly anchored in concrete, near-term implementations, ensuring immediate utility while building a foundation for long-term maintainability and scalability. The division of labor among Unreal Engine 5 for production, Godot for rapid prototyping, and Unity for experimentation is managed effectively through a shared set of Rust-based schemas and a central knowledge graph, ensuring functional consistency across disparate technology stacks. The strategic use of mathematical models, focused initially on rigorous validation and later repurposed for design guidance, provides a powerful, data-driven backbone for decision-making, eliminating ambiguity and reinforcing the project's core principles.

The synthesis of these elements reveals a project management philosophy that prioritizes verifiability, traceability, and automation above all else. The repository itself is treated as a first-class product, with its schemas, knowledge graphs, and CLI tools being version-controlled, tested, and maintained alongside the game's source code. This elevates the status of the development infrastructure to that of a critical asset. The GAMEMODE.ai system is not an afterthought but an integral part of this architecture; its effectiveness is predicated on the very structure the framework is designed to create—a clean, unambiguous, and highly contextualized environment for code generation [1]. The entire system is designed to reduce cognitive load by making the project's intent machine-readable and accessible through semantic queries, thereby accelerating development and lowering the barrier to entry for new contributors.

To transition this framework from a conceptual design to a fully functional system, a strategic, phased implementation roadmap is essential. The following priorities are recommended based on their impact on establishing the foundational infrastructure:

1. **Establish the Core Knowledge Graph Index:** The highest-impact initial action is to create the foundational KG index files, `knowledgegraphsystems.json` and `knowledgegraphfeatures.conker.json`. This involves manually cataloging the most critical systems and features already present in the repository's structure, such as the core UE5 classes (`CLUCharacterBase`, `CLUPlayerController`), the multiplayer game modes, and the primary Rust tools . This step is crucial as it provides the initial semantic map of the project, unlocking all downstream KG-based navigation and querying capabilities.

2. **Define and Enforce Critical Core Schemas:** Concurrently, the most vital data contracts should be formalized in JSON schema files. The `FileGenerationContract.schema.json` is paramount, as it will govern all

AI-generated code, ensuring it adheres to project conventions from the outset . Following this, schemas for `NetworkConstraints` and `MissionDAG` should be developed. These schemas will dictate the structure of the networking model and the objective flows for the Heist, War, and Alien Base modes, respectively, providing the contracts that the UE5, Godot, and Unity implementations must satisfy .

3. **Implement a Minimal Build Gate:** To demonstrate the value of the mathematical modeling approach, begin by implementing a single, high-value CI check. A VRAM budget validator is an excellent candidate. This involves creating a tool that parses asset manifests to calculate VRAM usage and integrating it into a GitHub Action that compares the result against a predefined maximum. Successfully failing a build due to a budget violation will prove the power of the validation-first approach and provide a template for future, more complex checks.

4. **Develop the Repository Indexing Tool:** The `repo_index_generator.rs` tool must be built to automate the population of the knowledge graph with granular detail . This tool will parse source files to extract symbols (classes, functions, variables) and enrich the KG nodes with this information. This automation is essential for scaling the KG's utility and keeping it synchronized with the rapidly evolving codebase.

5. **Refine AI Context and Prompts:** The `Docs/AI_Chat_Context` directory is the lynchpin for successful AI collaboration. Continuously refine the `System_Prompts_GAMEMODE_ai.md` and `File_Generation_Guidelines.md` documents to reflect the evolving capabilities of the KG and schemas [20]. As new schemas are introduced, they should be documented here so the AI has an accurate and up-to-date model of the project's expectations and constraints. This ensures that the AI's output remains aligned with the project's rigorous standards.

By executing this roadmap, the "Conker: Live & Uncut" project can systematically build out its advanced framework, transforming its repository into a highly organized, self-validating, and AI-friendly system. This will not only facilitate the recreation of the original game's vision but also create a powerful, reusable toolchain for future fan projects and game development endeavors.

## Reference


1. 1 Introduction - arXiv https://arxiv.org/html/2603.09400v1
2. [PDF] DRAFT Global Digital Citiverse Framework | ITU https://www.itu.int/metaverse/wp-content/uploads/2025/11/DRAFT-Global-Digital-Citiverse-Framework.pdf
3. Topics - Pub.dev https://pub.dev/topics
4. Black halter train multi-layered tulle dress No gloves ... - AliExpress https://www.aliexpress.com/item/1005009459484483.html
5. Claude Code Mistakes: How to Avoid Them | John Mikhail posted on ... https://www.linkedin.com/posts/john-mikhail_i-used-claude-code-all-the-wrong-ways-so-activity-7374168091905277952-M_Zk
6. Development Teams as a Service. - MindTrust https://mindtrust.com/development-teams/
7. JSON validation using github workflow - Stack Overflow https://stackoverflow.com/questions/67667472/json-validation-using-github-workflow
8. https://zenodo.org/records/6000983/files/Custom-La... https://zenodo.org/records/6000983/files/Custom-Labels.csv?download=1
9. ARTICLE IN PRESS - Nature https://www.nature.com/articles/s41598-026-45895-2_reference.pdf
10. EVMbench: Evaluating AI Agents on Smart Contract Security - arXiv https://arxiv.org/html/2603.04915v1
11. [PDF] Recent Developments in Machine Learning Methods for Stochastic ... https://hal.science/hal-03656245v1/file/ML_in_Control_and_Games____SIAM_review_HAL.pdf
12. [PDF] Formal Verification Using Static and Dynamic Analyses by ... https://cs.nyu.edu/media/publications/zaks_aleksandr.pdf
13. [PDF] arXiv:2411.05599v1 [cs.GT] 8 Nov 2024 https://arxiv.org/pdf/2411.05599
14. [PDF] From Program Verification to Program Synthesis - Microsoft https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/popl10_synthesis.pdf

15. Dictionary of Acronyms and Technical Abbreviations - Academia.edu https://www.academia.edu/64999604/Dictionary_of_Acronyms_and_Technical_Abbreviations
16. Designing Digital Humans with Ambient Intelligence - arXiv https://arxiv.org/html/2604.05120v1
17. [PDF] Computer Science Curricula 2023 - CS2023 https://csed.acm.org/wp-content/uploads/2023/03/Version-Beta-v2.pdf
18. User input in github actions (specify repo branch, etc) - Stack Overflow https://stackoverflow.com/questions/59429622/user-input-in-github-actions-specify-repo-branch-etc
19. Building AI Data Analysts with Claude Code: A Game Changer https://www.linkedin.com/posts/shaneausleybutler_its-been-a-few-days-since-i-started-testing-activity-7429531323318554625-cEOv
20. Findings of the Association for Computational Linguistics: EMNLP ... https://aclanthology.org/volumes/2025.findings-emnlp/
21. Findings of the Association for Computational Linguistics: ACL 2025 https://aclanthology.org/volumes/2025.findings-acl/
22. [PDF] Open RAN Security: Challenges and Opportunities - arXiv https://arxiv.org/pdf/2212.01510
23. Spaces - LtVT8pJjInrrHVCovzy PDF Export - Scribd https://www.scribd.com/document/692782956/Spaces-LtVT8pJjInrrHVCovzy-PDF-22534eafab9af424e9daed8d043bf7b798ae92d6ddc38e8709c69f91497827d8-Export
24. Add pattern validation in json schema when property is present https://stackoverflow.com/questions/63693345/add-pattern-validation-in-json-schema-when-property-is-present
25. GitHub Actions documentation https://docs.github.com/actions
26. OpenClaw Setup and PM Automation Guide for GitHub - LinkedIn https://www.linkedin.com/posts/aagupta_openclaw-has-245000-github-stars-and-zero-activity-7439531848130613248-Usj8
27. Boost Code Quality with Multi-Layered Security and CI/CD Workflows https://www.linkedin.com/posts/jpresley23_in-the-agentic-engineering-community-solving-activity-7442255701038567424-sY4K
28. [PDF] bhasha 2025 - ACL Anthology https://aclanthology.org/2025.bhasha-1.pdf
29. Computer Science - arXiv https://arxiv.org/list/cs/new
30. words_SG_upto2010.txt - Zenodo https://zenodo.org/record/5516252/files/words_SG_upto2010.txt

31. How to specify defaults conditionally with JSON Schema https://stackoverflow.com/questions/52992965/how-to-specify-defaults-conditionally-with-json-schema
32. [PDF] Lectures on Runtime Verification. Introductory and Advanced Topics https://inria.hal.science/hal-01762298/file/rv-book-editor-version.pdf
33. Theses & Reports - Computer Science https://cs.nyu.edu/dynamic/reports/?type=PhD&year=all
34. 333333 23135851162 the 13151942776 of 12997637966 https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
35. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
36. Ude My For Business Course List | PDF | Cloud Computing - Scribd https://www.scribd.com/document/652251737/Ude-My-for-Business-Course-List
37. How do I validate JSON using an existing schema file in Rust? https://stackoverflow.com/questions/44733603/how-do-i-validate-json-using-an-existing-schema-file-in-rust
38. A Formal Verification Study of Security Vulnerabilities in AI ... - arXiv https://arxiv.org/html/2604.05292v1
39. Arxiv今日论文| 2026-04-06 - 闲记算法 http://lonepatient.top/2026/04/06/arxiv_papers_2026-04-06
40. Artificial Intelligence - arXiv https://arxiv.org/list/cs.AI/new
41. 人工智能2026_4_6 - arXiv每日学术速递 http://www.arxivdaily.com/thread/78473
42. Machine Learning and Knowledge Discovery in Databases https://link.springer.com/content/pdf/10.1007/978-3-031-26412-2.pdf
43. Simple Index - SUSTech Open Source Mirrors https://mirrors.sustech.edu.cn/pypi/simple/
44. VSCode, how to check workflow syntax for GitHub Actions https://dev.to/robertobutti/vscode-how-to-check-workflow-syntax-for-github-actions-4k0o
45. [PDF] arXiv:2505.23628v1 [cs.CL] 29 May 2025 https://arxiv.org/pdf/2505.23628?
46. (PDF) Construction of Knowledge Graphs: State and Challenges https://www.researchgate.net/publication/368716633_Construction_of_Knowledge_Graphs_State_and_Challenges
47. Autonomous Knowledge Graph Construction | PDF - Scribd https://www.scribd.com/document/975100136/AutoschemaKG-2505-23628v3
48. [PDF] The 10th Workshop on Financial Technology and Natural Language ... https://aclanthology.org/2025.finnlp-2.pdf

49. Artificial Intelligence and Security - Springer Nature https://link.springer.com/content/pdf/10.1007/978-981-15-8083-3.pdf
50. Simple Index - Alibaba Cloud https://mirrors.aliyun.com/pypi/simple/
51. download as TXT - DistroWatch.com https://distrowatch.com/resource/nixos/nixos-unstable.txt